Accumulator-based Forward-Secure Decoupled Distributed Authentication Scheme in Multihop IoT Networks

Sreeparna Das, Abhishek Tiwari, Anup Kulkarni, Manas Khatua

***Abstract*—A secure and lightweight authentication scheme is essential for resource-constrained multihop Internet of Things (IoT) networks. Existing distributed schemes rely on parent IoT nodes to authenticate their newly joined child nodes. Thus, certain parent nodes with a large number of child nodes and their authentication requests impose high resource overhead on themselves, leaving other nodes with fewer or no child underutilized. This causes a skewed authentication load scenario in the network. Therefore, this paper proposes a new decoupled distributed authentication scheme, decoupling the authentication from node joining process. It is designed by leveraging the idea of a one-way membership hash function. At the end of authentication, secured session key is shared among the new node, authenticator, and the root node or gateway in multihop IoT network for achieving security in data communication. However, the shared session key is dependent on long-term secrets, making it vulnerable to future exposure and violating Perfect Forward Secrecy. To mitigate this limitation, a session key update phase is incorporated in the proposed scheme. The proposed scheme is evaluated via ProVerif-based security analysis, theoretical and simulation-based performance analysis. Hardware implementation confirms that the proposed scheme reduces energy and CPU time by 33.33% and 45.45%, respectively, over the best performing state-of-the-art, LightSAE.**

***Index Terms*—Lightweight, Authentication, multihop, Forward Secrecy and IoT.**

I. Introduction

THE Internet of Things (IoT) interconnects millions of smart devices, facilitating data sensing, gathering, and transfer to a gateway or cloud server [1]. In a multihop IoT network, the data is communicated between IoT devices and the gateway or cloud server through a multihop path made of IoT nodes [2]. Extensively adopted in industrial application scenarios [3], such IoT networks are usually installed in outdoor and unattended environments, exposing them to security vulnerabilities. Therefore mandates lightweight authentication before initiating a communication session [4] of IoT devices with limited resources [5].

Most IoT authentication schemes follow a centralized design, where either a cloud server or a central server acts as an authenticator that authenticates IoT nodes [6], [7], [8], incurring significant computational and communication overhead in multihop communication of authentication packets. Therefore, distributed authentication mechanisms are explored, which allow multiple entities to authenticate other IoT nodes in a multihop IoT network. In such a distributed scheme proposed in LightSAE [9], IoT nodes are assigned the responsibility of authenticating their newly joined child nodes. However, the selection of parent nodes is performed by their child nodes at the time of joining. This scenario leads to the flooding of certain IoT nodes, which have multiple child nodes, with authentication requests. Meanwhile, nodes with fewer or no child nodes remain underutilized. The creation of authentication hotspots around certain parent nodes causes a skewed distribution of authentication load in the network. In addition, LightSAE leverages a static secret key to conduct all authentication processes, necessitating its storage. This increases its vulnerabilty to leakage, compromising authentications. Moreover, the use of the same key in LightSAE across multiple hops and authentications introduces predictability to authentication messages.

On the other hand, peer-to-peer (P2P) authentication schemes [10], [11] involve an IoT node to authenticate another IoT node before they can establish a secure communication with each other, while both may or may not have a parent-child relationship. However, IoT nodes that act as a primary communication point for other IoT nodes are subjected to a high volume of authentication requests from them before the initiation of communication. On the other hand, IoT nodes with fewer or no communication interaction with other nodes contribute minimally in terms of authentication responsibilities. Such uneven distribution of authentication tasks creates a skewed authentication load scenario in the network. Moreover, these schemes rely on the execution of complex cryptographic operations, such as Elliptic Curve Cryptography (ECC) and encryption/decryption, which imposes significant computational burden on IoT nodes. Alongside this, these schemes require the transmission of large cryptographic messages, such as ECC parameters, producing increased communication costs and thereby causing higher authentication delay and energy requirement.

To mitigate the drawbacks, a lightweight and decoupled distributed authentication scheme is proposed where both parent and non-parent IoT nodes authenticate newly joined IoT nodes. In this way, the decoupled architecture is introduced, separating/decoupling processes of node joining and authentication through a pre-selected set of authenticators among the IoT nodes in the network. It enables a selected IoT node from the set to authenticate the newly joined node. To further enhance security, the scheme integrates Physically Unclonable Func-

The authors are with the Department of Computer Science and Engineering, Indian Institute of Technology Guwahati, Assam 781039, India (E-mail: d.sreeparna@iitg.ac.in; abhishekkt@iitg.ac.in; anup.kulkarni@iitg.ac.in; manaskhatua@iitg.ac.in).

tion (PUF) generated responses, and a session-based random. Furthermore, the incorporation of a lightweight hash-based accumulator [12] and XOR operations, alongside two encryption/decryption operations, makes the scheme lightweight for resource-constrained IoT nodes. The hash-based one-way accumulator further helps the authenticator verify the newly joined node's membership without revealing its secret. Upon completion of the authentication, the authenticator securely notifies the gateway about the newly joined node, facilitating session key sharing with the gateway and newly joined node. However the shared session key is established using long-term secrets, protecting communications between them for an extended duration. The future exposure of those long-term secrets leads to the compromise of the session key and, in turn, the communications protected by that key. This facilitates the violation of Perfect Forward Secrecy (PFS) concerning those communications. Thus, a session key update phase is incorporated in the proposed scheme, where one-time enrolment is followed by periodic execution of authentication and session key sharing phase. The extended version of the proposed scheme additionally updates the session keys independently of the long-term secrets through the session key update phase. This phase is executed before every communication between the gateway and the node until the need for re-execution of the node authentication and session key sharing phase. Thereby reduces the impact of long-term secret compromise and maintains PFS. The initial work is published in [13]. Thus the additional differences between the previous and the present work are as follows. **i)** A session key update phase is incorporated with the proposed scheme. **ii)** Hardware implementation is performed for the proposed scheme. **iii)** Security analysis is performed for the session key sharing and update phase along with the enrolment and authentication phases using ProVerif. **iv)** Two additional state-of-the-art schemes, SAKMS [10] and SensLi [11], are compared with the proposed scheme.

The major contributions of the entire paper are outlined as follows.

- A lightweight, decoupled distributed authentication scheme is proposed for multihop IoT networks, along with session key sharing between the gateway and the newly joined IoT node via the authenticator.
- A session key update phase is incorporated into the proposed scheme where the session key between the gateway and that IoT node is updated before every communication. The updated session keys are independent of long-term secrets, maintaining PFS.
- The security of the proposed scheme is verified through ProVerif.
- A comprehensive performance analysis is performed using theoretical analysis, Cooja simulation and hardware implementation.

The remainder of the paper is structured as follows. Section II surveys the existing literatures. Section III outlines the proposed scheme, including the node enrolment, the node authentication, and the session key sharing phase. The proposed scheme integrated with the session key update phase is demonstrated in Section IV. Security analysis is presented in Section V, which is followed by the performance analysis in Section VI. In Section VII, the hardware implementation results are showcased. Finally, the paper is concluded in Section VIII.

II. Literature Survey

Most traditional centralized authentication schemes [6], [7], [8], [14], [15] relies on a centralized authenticator, leading to an increased computational and communication overhead in a multihop network. Distributed authentication schemes helps in mitigating the drawback of centralized schemes through multiple authenticators. However, scheme, [16], [17], suffers from skewed authentication load on certain edge servers or fog servers and ignores multihop IoT consideration. In LightSAE [9], parent IoT nodes authenticate child nodes, causing a skewed authentication overhead on nodes with many children, while other with fewer or no children remain underutilized. On the other hand, traditional peer-to peer (P2P) schemes [18], [19], [20] depend on long-term key storage, causing security vulnerabilities. It is mitigated by PUF-based P2P schemes [11], as it avoids key storage, but suffers from skewed authentication overhead on certain IoT nodes. Both [11], [10] suffers from high computational and communication overhead due to computationally intensive Elliptic Curve Cryptography (ECC) operations.

Therefore, this paper proposes a lightweight decoupled authentication scheme, where both parent and non-parent can authenticate newly joined node, mitigating skewed authentication load scenario. It further uses a hash-based one-way accumulator to verify the newly joined node's membership by the authenticator, performed without revealing node's secret. However, the session key, shared initially between the gateway and the node, depends on long-term secrets, violating the Perfect Forward Secrecy (PFS) concerning these communications. To mitigate, an additional session key update phase, based on ephemeral keys, is incorporated with the proposed scheme.

As shown in Table I, the proposed scheme is compared with state-of-the-art schemes, LightSAE [9], SAKMS [10], and SensLi [11]. The state-of-the-art schemes either depends on computationally heavy ECC operations or long-term key storage. All of them suffer from a skewed authentication load scenario. These drawbacks are mitigated by the proposed scheme DAuth, but it does not have PFS. It is mitigated by incorporating session key update phase to DAuth, introducing DAuthP.

III. Proposed Scheme DAuth

In an industrial automation system, an IoT device communicates with the gateway through multihop communication. To secure the communication, a lightweight and decoupled distributed authentication scheme, DAuth, is proposed. Ath, one of a set of designated IoT nodes ($Ath^{'} = \{Ath_1, Ath_2, \ldots, Ath_m\}$, such that $Ath \in Ath^{'}$) acts as an authenticator to the newly joined node N. Node N has network address and the unique identifier (ID_{Ath}) of its designated authenticator. The gateway (GW) is initially considered to have selected the set of authenticators, where GW have a

TABLE I
COMPARISON OF FEATURES ACROSS AUTHENTICATION SCHEMES.

Scheme	Dec	MSO	SLS	UAS	PFS	COH	MOH
SAKMS [10]	✓	×	×	×	✓	High	Low
SensLi [11]	✓	×	✓	✓	✓	High	Low
LightSAE [9]	×	×	×	×	✓	High	High
DAuth	✓	✓	✓	✓	×	Low	Low
DAuthP	✓	✓	✓	✓	✓	Medium	Medium

Legend: ✓= Achieved, × = Not Achieved.
Dec: Decoupled authentication.
MSO: Mitigation of skewed authentication overhead.
SLS: Storage of long-term secret.
UAS: Updation of authentication secrets protecting authentication messages.
PFS: Ephemeral key-based session key generation maintaining PFS.
COH: Computational overhead — Low (Hash/XOR/2 symmetric encryption–decryption),
Medium (>2 symmetric encryptions/
decryptions), High (ECC/Pairing operations).
MOH: Message exchanges/Communication overhead — Low (1–3), Medium (4–5), High (>5).

TABLE II
NOTATIONS USED.

Notation	Description
N, Ath	Newly joined node and its authenticator
GW	Gateway
$H(\cdot)$	Collision-resistant hash function
$SE(\cdot)$	Symmetric encryption
$SD(\cdot)$	Symmetric decryption
c_N	Challenge of node N
m_N	Session-based random for node N
y_N	Unique secret of node N
Y_N	$H(y_N)$
c_{Ath}	Challenge of authenticator Ath
T_{AccAth}	Accumulated value in the authenticator
$K_{GW,N}$	Shared session key among the GW and N
$K_{GW,Ath}$	Shared secret key among the GW and Ath
t_{auth}	Authentication completion timestamp of N

shared secret key with each one of them, that is $K_{GW,Ath}$. The authenticator may or may not be the parent node of the newly joined node. DAuth is executed in three phases: node enrolment phase, node authentication phase, and session key sharing phase. The frequently used notations adopted in the scheme are listed in Table II.

A. Node Enrolment Phase

The node enrolment phase is a one-time event performed by the newly joined node N to enrol its node-specific secret y_N into the accumulator value T_{AccAth} maintained at the authenticator (Ath) through a secure channel. It is performed through a hash-based accumulator mechanism [12], which combines multiple hashed values ($Y_N = H(y_N)$) into the single accumulator value (T_{AccAth}). Thus enables the membership verification of node N to the authenticator Ath without revealing y_N or Y_N in the subsequent authentication phase. PUF-based responses and session-based random are also exchanged in this phase, ensuring the subsequent secure communications of y_N. Figure 1 depicts a detailed illustration of the node enrolment process.

Initially, the authenticator Ath does not have any information about the node N. So at first, N sends its unique identifier ID_N along with the Registration Request to Ath, initiating enrolment. Ath responses with a challenge c_N and a session-based random m_N. Using PUF_N and the c_N, N computes

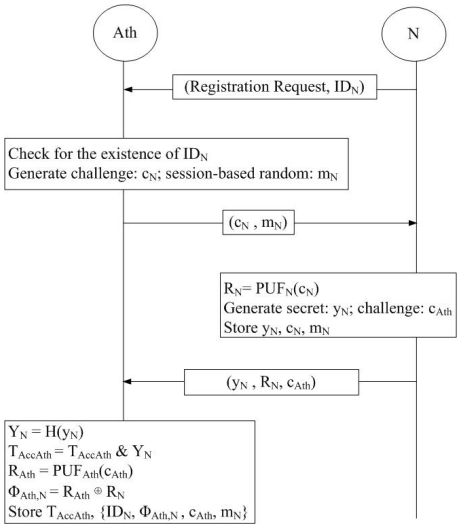


Fig. 1. Node Enrolment Phase through secure channel.

R_N and generates its unique secret y_N, then securely sends y_N, R_N, a challenge for Ath i.e. c_{Ath}. Ath hashes y_N to generate Y_N and performs bitwise AND operation between the accumulator value T_{AccAth} and Y_N, updating T_{AccAth}. In this way Ath enrols N. To secure the long-term secret R_N, Ath computes R_{Ath} ($PUF_{Ath}(c_{Ath})$) and calculates Equation (1).

$$\Phi_{Ath,N} = R_{Ath} \oplus R_N \tag{1}$$

B. Node Authentication Phase

The node authentication phase proves node N's membership to the authenticator Ath through an open channel, without revealing its unique secret y_N or Y_N. This is achieved by securely transmitting Y_N utilizing PUF-based response R_N and session-based random m_N, shared during enrolment phase. The flow is presented in Figure 2.

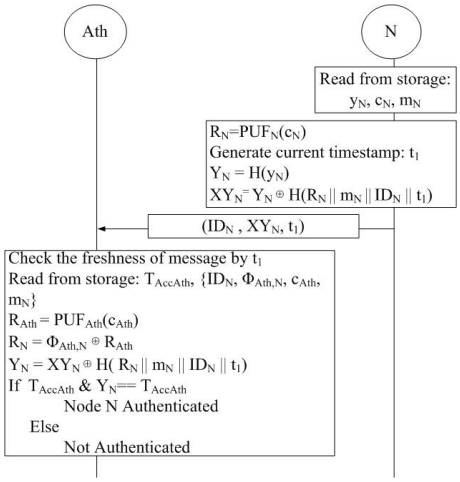


Fig. 2. Node Authentication Phase.

The phase is initiated with node N transmitting $\{ID_N, XY_N, t_1\}$. Here, XY_N is secured version of its hashed secret Y_N, generated using the hash of R_N, m_N, its unique

identifier ID_N, and the current timestamp t_1. The operation is presented in Equation (2).

$$XY_N = Y_N \oplus H(R_N \parallel m_N \parallel ID_N \parallel t_1) \tag{2}$$

Upon receiving the $\{ID_N, XY_N, t_1\}$, Ath first verifies the existence of N in its records, reading it from its storage as $\{ID_N, \Phi_{Ath,N}, c_{Ath}, m_N\}$. It also checks for the freshness of the message, using t_1. After this Ath recomputes R_N using Equation (3).

$$R_N = \Phi_{Ath,N} \oplus R_{Ath} \tag{3}$$

Using the reconstructed R_N, m_N, ID_N, and the received t_1, Ath extracts Y_N from the received XY_N, as shown in Equation (4).

$$Y_N = XY_N \oplus H(R_N \parallel m_N \parallel ID_N \parallel t_1) \tag{4}$$

Y_N is then used by Ath to verify the membership of N, applying bitwise AND operation between Y_N and the stored accumulator value T_{AccAth} to produce the new value. If the new value matches T_{AccAth}, node N is successfully authenticated.

C. Session Key Sharing Phase

After the successful authentication, Ath securely shares the session key $K_{GW,N}$ with GW and N along with a new session-based random M_{new} for unpredictability. It is initiated with communication of M_{new} in the form of XM_{new} to N, using Y_N, m_N, R_N, ID_{Ath}, ID_N, and the current timestamp t_2. It is shown in Equation (5).

$$XM_{new} = M_{new} \oplus H(Y_N \parallel m_N \parallel R_N \parallel ID_{Ath} \parallel ID_N \parallel t_2) \tag{5}$$

Ath then constructs $EK_{GW,N}$ that securely encapsulates the session key $K_{GW,N}$ ($H(R_N||M_{new})$) to be shared with GW and N and the current timestamp or the authentication completion time t_{auth}, using the pre-shared key ($K_{GW,Ath}$) between GW and Ath. The computation of $EK_{GW,N}$ is shown in Equation (6). The message $\{ID_{Ath}, ID_N, EK_{GW,N}\}$ is then sent to GW by Ath and $\{XM_{new}, t_2\}$ to N.

$$EK_{GW,N} = SE\big(K_{GW,Ath}, \big(K_{GW,N} \parallel ID_{Ath} \parallel ID_N \parallel t_{auth}\big)\big) \tag{6}$$

From $\{ID_{Ath}, ID_N, EK_{GW,N}\}$, GW obtains the knowledge of N, its authenticator Ath and the session key $K_{GW,N}$, decrypting $EK_{GW,N}$ with $K_{GW,Ath}$, as shown in Equation (7).

$$K_{GW,N}, ID_{Ath}, ID_N, t_{auth} = SD\big(K_{GW,Ath}, EK_{GW,N}\big) \tag{7}$$

GW updates the record of ID_N, storing the identifier of its authenticator ID_{Ath}, along with the session key $K_{GW,N}$ and t_{auth}.

On the other hand, N validates the freshness of the received message $\{XM_{new}, t_2\}$ at first, then retrieves the new session-based random M_{new}. The operation is presented in Equation (8).

$$M_{new} = XM_{new} \oplus H(Y_N \parallel m_N \parallel R_N \parallel ID_{Ath} \parallel ID_N \parallel t_2) \tag{8}$$

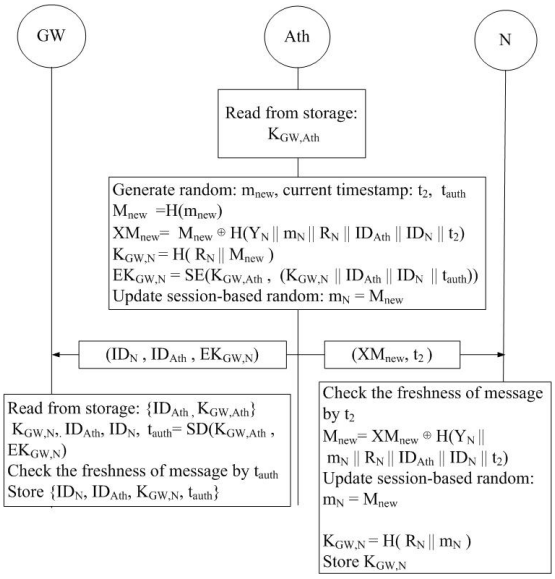


Fig. 3. Session Key Sharing Phase.

As a result, Ath and N both update their shared session-based random m_N with M_{new}. The entire flow of the session key sharing phase is depicted in Figure 3.

IV. DAuth with Perfect Forward Secrecy

This section states the shortcoming of DAuth, that is the violation of Perfect Forward Secrecy (PFS). It also incorporates a session key update phase to mitigate the shortcoming, introducing DAuthP.

A. Shortcoming of DAuth

According to DAuth, the gateway GW and the newly joined node N share a session key $K_{GW,N}$. Before the initiation of secure communication with N, the GW checks validity of $K_{GW,N}$ with the help of t_{auth} retrieved from the record of ID_N. If the difference between the current timestamp t_c and t_{auth} lies within the threshold t_δ, as shown in Equation (9), $K_{GW,N}$ is valid. Otherwise, GW directs N for re-authentication to generate a new session key with an updated session-based random m_N. Until the re-authentication, $K_{GW,N}$ remains unchanged, ensuring secure communication between GW and N for this duration.

$$t_{auth} - t_c \leq t_\delta \tag{9}$$

If long-term secrets, R_N and m_N (long-term for a duration of t_δ), are leaked in the future, they compromise the session key, made of the same R_N and m_N. This, in turn, compromises all communications between GW and N, which are secured by that session key during the duration of t_δ. This facilitates the violation of Perfect Forward Secrecy (PFS) concerning these communications. In the extended version of DAuth, called DAuthP, this drawback is mitigated by incorporating a session key update phase, which generates a fresh $K_{GW,N}$ before the initiation of each communication between GW and N. The fresh $K_{GW,N}$ is made of unique

private parameters, a_{GW} and b_N, which are generated and discarded during every session. Thus, the session key is not dependent on long-term R_N and m_N. Therefore, any future leakage of R_N and m_N is not going to compromise the already computed session keys and the communications encrypted by those keys.

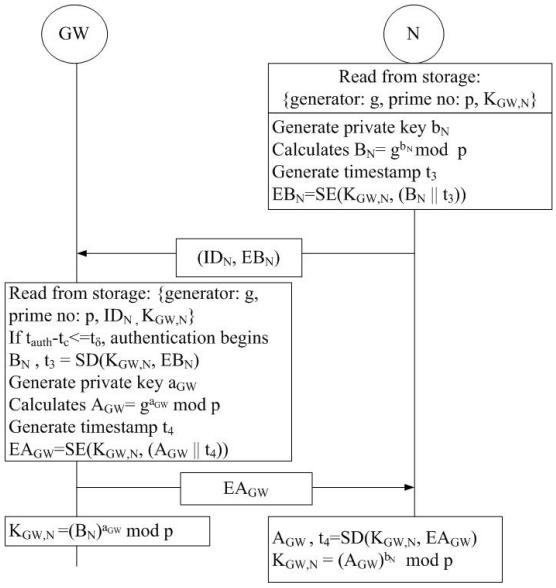


Fig. 4. Session Key Update Phase.

B. Proposed Session Key Update Phase

The shared session key $K_{GW,N}$ is refreshed by this new phase, enhancing the proposed DAuth. This step is important to generate a unique session key before each communication between GW and N, independent of long-term secrets R_N and m_N, incorporating PFS. The refreshment of the session key $K_{GW,N}$ is performed using an ephemeral Diffie Hellman key exchange, as illustrated in Figure 4. It is assumed that each node N, including GW, is embedded with two parameters: a generator g and a prime number p. During each session, the node N produces a random private value b_N, computing public parameter B_N, as mentioned in Equation (10).

$$B_N = g^{b_N} \ mod \ p \tag{10}$$

This B_N is securely transmitted along with the current timestamp t_3 to GW, as N encrypts both of them using the current session key $K_{GW,N}$, producing EB_N, as mentioned in Equation (11).

$$EB_N = SE(K_{GW,N}, (B_N \parallel t_3)) \tag{11}$$

On receiving $\{ID_N, EB_N\}$, GW first checks the entry of ID_N, from where it obtains the current session key $K_{GW,N}$ and t_{auth}. At first, it verifies the validity of $K_{GW,N}$ using t_{auth}. As shown in Equation (9), if the difference between t_{auth} and the current timestamp t_c is not within the threshold of t_δ, then GW aborts the session key update phase and redirects N for re-authentication with Ath.

If the difference between t_{auth} and t_c is within the threshold, GW decrypts EB_N using the present $K_{GW,N}$ and verifies the freshness of the message inside the decrypted EB_N using timestamp t_3. After the freshness confirmation, GW generates its private value a_{GW}, which is used to compute its public parameter A_{GW}, as performed in Equation (12).

$$A_{GW} = g^{a_{GW}} \ mod \ p \tag{12}$$

This A_{GW} is sent back to N along with the current timestamp t_4 in an encrypted message. The encryption is performed by GW using the current session key $K_{GW,N}$ to generate EA_{GW}. The computation is performed in Equation (13).

$$EA_{GW} = SE(K_{GW,N}, (A_{GW} \parallel t_4)) \tag{13}$$

N decrypts the received EA_{GW} to obtain A_{GW}. At this point, both GW and N have public parameters B_N and A_{GW}, respectively, for the final session key generation. Using Equation (10), GW computes the fresh session key using its private value a_{GW} as shown in the following Equation (14).

$$\begin{aligned} K_{GW,N} &= (B_N)^{a_{GW}} \ mod \ p \\ &= g^{b_N * a_{GW}} \ mod \ p \end{aligned} \tag{14}$$

On the other hand, N derives the same session key $K_{GW,N}$ by exponentiating the obtained public parameter A_{GW} with its private value b_N, using Equation (12). The computation is presented in the following Equation (15).

$$\begin{aligned} K_{GW,N} &= (A_{GW})^{b_N} \ mod \ p \\ &= g^{a_{GW} * b_N} \ mod \ p \end{aligned} \tag{15}$$

In accordance with the commutative property of exponentiation modulo a prime, both values of $K_{GW,N}$ in Equation (14) and Equation (15) arrives at the same value. This whole operation is performed by GW and N, without the transmission of their private values in the network.

V. Security Analysis

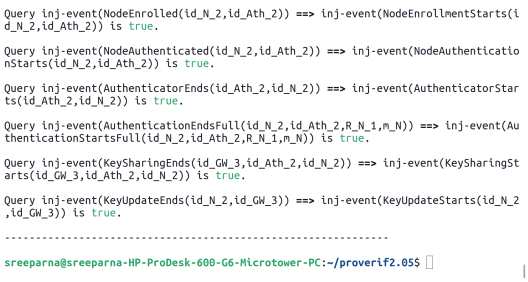
A ProVerif-based analysis [21] is conducted to verify resilience of the proposed DAuth and DAuthP against standard attacks, such as replay and man-in-the-middle, following Dolev-Yao [22] adversary model. For the formal security validation of the schemes explained in Section III and IV, they are translated into ProVerif queries.

A. Completion of Authentication and Key Generation, along with Resistance to Replay and Man-in-the-Middle Attack

This section, as shown in Figure 5a verifies the correctness of the proposed DAuth and DAuthP across the newly joined node N, the authenticator Ath, and the gateway GW through Identity and Full agreement. They ensure the consistency of identities and session parameters with the participating entities. If Equations (16) and (17) are true then the session key sharing and update phases are performed by the legitimate GW, Ath, and N, guaranteeing resilience against replay and man-in-the-middle attacks.

$$\texttt{inj-event}(\texttt{KeySharingEnds}(ID_{GW}, ID_{Ath}, ID_N)) \Rightarrow \texttt{inj-event}(\texttt{KeySharingStarts}(ID_{GW}, ID_{Ath}, ID_N)) \tag{16}$$

$$\texttt{inj-event}(\texttt{KeyUpdateEnds}(ID_N, ID_{GW})) \Rightarrow \texttt{inj-event}(\texttt{KeyUpdateStarts}(ID_N, ID_{GW})) \tag{17}$$



(a) Completion of Authentication and Key Generation

(b) Reachability and Secrecy of Session Key

Fig. 5. ProVerif Output.

B. Reachability and Secrecy of Session Keys

An additional security objective is to confirm the confidentiality of the session key $K_{GW,N}$, shared and updated with N and GW. In Figure 5b, if queries output *false*, it ensures that the adversary cannot procure either the session key $K_{GW,N}$ (both shared and updated) or messages, `SecretK_GW_N_N[]`, `SecretK_GW_N_Ath[]`, and `SecretK_GW_N_Update[]`, encrypted by them.

VI. Performance Analysis

A. Theoretical Performance Analysis

Theoretical performance analysis is essential to assess the efficiency and feasibility of authentication schemes before they are implemented in a simulator and a hardware platform under uniform assumptions. The PyCrypto [23] library is used on a Linux platform to quantify the computational cost of various cryptographic operations, as mentioned in Table III. The results reveal that schemes like SAKMS [10], and SensLi [11] impose higher computational overheads of 4.20 ms, and 2.53 ms, respectively. Whereas both DAuth and DAuthP preserve efficiency with a computational cost of 0.56 ms and 1.03 ms. In Table IV, the communication cost of DAuth and DAuth is least among the state-of-the-art schemes.

TABLE III
Comparison of Computational Cost: Authentication, Key Sharing and Key Update Phase

Scheme	Operations	Cost (ms)
SAKMS [10]	$6T_{hash} + 4T_{ecc}$	4.20
SensLi [11]	$2T_{puf} + 12T_{hash} + 2T_{ecc} + 4T_{rand}$	2.53
LightSAE [9]	$16T_{aes} + 4T_{exp}$	1.33
DAuth	$2T_{puf} + 8T_{hash} + 2T_{aes} + 1T_{rand}$	0.56
DAuthP	$2T_{puf} + 8T_{hash} + 6T_{aes} + 1T_{rand} + 4T_{exp}$	1.03

Benchmark (avg. of 100 iterations):
T_{rand} = 0.0018ms (Random generation), T_{puf} = 0.0522ms (PUF operation), T_{exp} = 0.0304ms (Modular Exponentiation), T_{hash} = 0.0353 ms (SHA-256), T_{aes} = 0.0867 ms (AES), T_{ecc} = 0.9973 ms (ECC Mult. / Add).

TABLE IV
Comparison of Communication Cost: Authentication, Key Sharing and Key Update Phase

Scheme	Number of Messages	Cost (bits)
SAKMS [10]	2	2612
SensLi [11]	3	4160
LightSAE [9]	6	1664
DAuth	3	928
DAuthP	5	1472

All schemes assume: temporary IDs and timestamps = 32 bits; random nonces, hash digests, challenge, responses, and symmetric ciphertexts = 256 bits; ECC points (P-256) = 512 bits

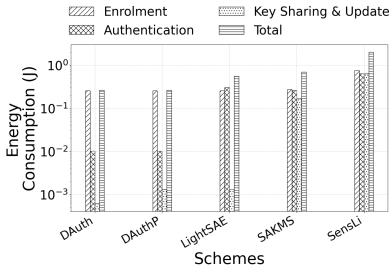
B. Simulation-Based Performance Analysis

Simulation-based performance analysis is important as it evaluates the proposed authentication schemes by considering the impact of the message communication on performance metrics, such as energy and CPU time, in a multihop network.

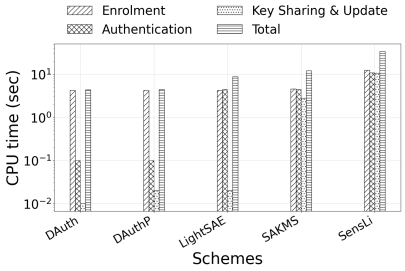
A comprehensive simulation-based performance analysis is performed, assessing the performance efficiency of the proposed DAuth, its extended version with PFS (DAuthP), and few baseline schemes, such as SensLi [11], SAKMS [10], and LightSAE [9]. The simulation is executed in the Cooja simulator using Cooja mote inside the Contiki-NG environment, where the CoAP protocol is utilized for the communication. A 100-node network is randomly deployed as the simulation scenario. The simulation time is 4920 seconds and are repeated 10 times. The propagation model in the simulation is UDGM (Distance Loss). In this network, 80 nodes are deployed as CoAP servers. One of these 80 nodes is considered to be the gateway or the Certificate Authority, depending on its role, which varies across the authentication schemes implemented. Out of the remaining 79 CoAP servers, a subset operates as authenticators. The other 20 nodes of the network are considered to be newly joined nodes, functioning as CoAP clients. This consideration is conducted to introduce simultaneous authentication requests on the authenticator without congesting the network.

1) Performance Analysis under Different Phases: In Figures 6a and 6b, the energy consumption and CPU time have been analyzed for the enrolment, authentication, and session key sharing phases, respectively. This is performed to analyze whether the proposed schemes, DAuth and DAuthP, are lightweight in comparison to LightSAE, SAKMS, and SensLi. Only in the case of DAuthP, both session key sharing and session key update phases are considered in the session key sharing analysis. The simulation is performed with two CoAP servers as authenticators, and the authentication load of 20 nodes is distributed equally among the two authenticators.

In case of the enrolment phase, both DAuth and DAuthP obtains the same energy requirement and CPU time as the best performing state-of-the-art scheme, LightSAE. In the authentication phase, both DAuth and DAuthP reduces the energy consumption and CPU time by 96.6% and 97.7%, respectively, with respect to LightSAE. When the session key sharing phase is considered, DAuth reduces the energy requirement and CPU time by 53.8% and 50% with respect to the same LightSAE. On the other hand, DAuthP obtains the same energy requirement and CPU time with respect to the

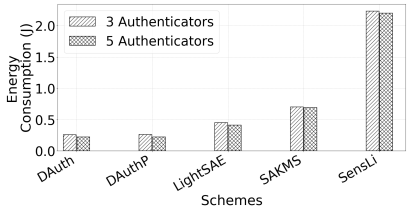


(a) Average Energy Consumption

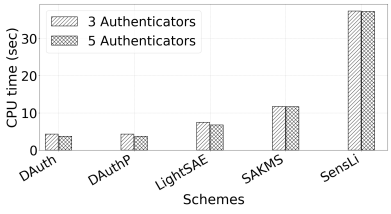


(b) Average CPU Time

Fig. 6. Performance analysis considering two authenticators and having equal authentication load.



(a) Average Energy Consumption



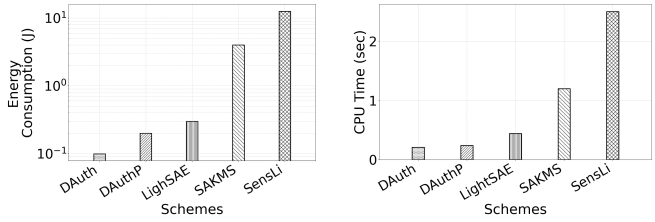
(b) Average CPU Time

Fig. 7. Performance analysis under skewed authentication load distribution.

best performing scheme, LightSAE.

In cumulative analysis, which includes enrolment, authentication, and session key sharing phase, DAuth reduces the energy and CPU time by 52.5% and 50.11% in comparison to the best performing scheme, LightSAE. In case of DAuthP, the energy and CPU time is reduced by 52.41% and 49.42% when compared with the same LightSAE.

2) Performance Analysis under Skewed Authentication Load Distribution: As mentioned in Figures 7a and 7b, the performance analysis is performed to demonstrate the impact of skewed authentication load distribution on the proposed schemes, DAuth and DAuthP, in comparison with the state-of-the-art schemes, LightSAE, SAKMS, and SensLi. The simulation is performed by varying the number of authenticators, considering two scenarios. One scenario considers 3 authenticators, in which 10% of 20 newly joined nodes are authenticated by one authenticator, 30% by the second authenticator, and 60% by the third authenticator. In the second scenario, 5 authenticators are responsible for the authentication of 20 newly joined nodes. In this scenario, 5% of authentication load is handled by the first authenticator, 15% by the second, 35% by the third, 20% by fourth authenticator, and the remaining by the fifth authenticator. However, it is important to mention that the proposed schemes are not limited to such distributions of nodes only. The above configuration has been used just as a representative example.



(a) Average Energy Consumption (b) Average CPU Time

Fig. 8. Performance analysis for Hardware Implementation.

In the scenario with 3 authenticators, DAuth and DAuthP, reduce the energy requirement by 43% and 42.2%, in comparison to the best performing state-of-the-art scheme, LightSAE. In case of 5 authentication, DAuth and DAuthP, achieve the reduction in the energy requirement by 46.34% and 45.85%, respectively, when compared with the same, LightSAE.

In case of the CPU time, for 3 authenticators, DAuth and DAuthP, reduce it by 42.8% and 42.5%, in comparison with the best performing scheme, LightSAE. On the other hand, for 5 authenticators, DAuth and DAuthP, achieves the reduction in the same by 45.72% and 45.3%, respectively, when compared with the same best performing scheme, LightSAE.

VII. Hardware Implementation

Hardware-based performance analysis plays a crucial role as it validates the feasibility of the proposed authentication schemes by implementing them in physical IoT devices, incorporating the real hardware environment conditions.

The proposed DAuth, DAuthP, and the baseline authentication schemes SensLi [11], SAKMS [10], and LightSAE [9] are implemented on a Raspberry Pi 4 Model platform. It is equipped with a quad-core 64-bit ARM cortex A-72 processor operating at 1.5 GHz. A uniform hardware setup is maintained with respect to all compared authentication schemes, ensuring a fair and consistent comparison analysis of energy consumption and CPU time. The implementation setup considers two Raspberry Pi platforms as IoT nodes. One of them performs the role of a newly joined node, initiating the authentication process. Another one functions as its authenticator, performing the necessary verification for authentication and key exchange. In addition, a Linux-based computer system is leveraged as the gateway or certificate authority, depending on its operational role, which varies across different authentication schemes implemented. Each of

the three entities maintains direct communication with each other.

As illustrated in Figure 8a, the proposed DAuth achieves the least energy consumption and reduces it by 50% with respect to its version with PFS, DAuthP. When compared with the best performing state-of-the-art scheme, LightSAE, both DAuth and DAuthP achieve an average reduction in energy consumption of 66.67% and 33.33%, respectively.

The CPU time results, as shown in Figure 8b, demonstrate the efficiency of both DAuth and DAuthP. In comparison with the best performing scheme, LightSAE, both DAuth and DAuthP reduces the CPU time by 52.3% and 45.45%, respectively.

The inconsistency between the hardware and simulation-based results arises due to the disparity in scale, where a 100-node network is assumed in simulation and for the hardware setup only two Raspberry Pis are used.

VIII. Conclusion

This paper proposes a lightweight, decoupled, and distributed authentication scheme, which allows both parent and non-parent nodes to authenticate newly joined nodes, mitigating the skewed authentication overhead. It uses a hash-based one-way accumulator for the verification of the node by the authenticator. As, the session key initially shared between the gateway and the newly joined node relies on long-term secrets, it violates the PFS. A session key update phase based on ephemeral private keys is incorporated to DAuth to overcome the PFS limitation, introducing DAuthP. ProVerif-based security analysis, theoretical, simulation and hardware-based performance analysis proves the efficiency of the proposed DAuth and DAuthP when compared with the state-of-the-art schemes, LightSAE, SAKMS, and SensLi.

References

[1] I. Zhou, I. Makhdoom, N. Shariati, M. A. Raza, R. Keshavarz, J. Lipman, M. Abolhasan, and A. Jamalipour, "Internet of Things 2.0: Concepts, Applications, and Future Directions," *IEEE Access*, vol. 9, pp. 70961–71012, 2021.

[2] B. Kang, D. Kim, and H. Choo, "Internet of Everything: A Large-Scale Autonomic IoT Gateway," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 3, pp. 206–214, 2017.

[3] N. Lajnef, A. Mami, and F. Derbel, "Applications of Wireless Sensor Networks and IoT Frameworks in Industry 4.0: A Systematic Literature Review," *Sensors*, vol. 22, no. 6, 2022.

[4] J. Granjal, E. Monteiro, and J. S. Silva, "Security in the Integration of Low-Power Wireless Sensor Networks with the Internet: A Survey," *Ad Hoc Networks*, vol. 24, pp. 264–287, 2015.

[5] M. Khatua, S. Das, and P. M. Rana, "PUF-based Lightweight Mutual Authentication Scheme for IoT-based Smart Grids," in *Proceedings of 2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, pp. 1–6, IEEE, 2024.

[6] S. A. Sheik and A. P. Muniyandi, "Secure authentication schemes in cloud computing with glimpse of artificial neural networks: A review," *Cyber Security and Applications*, vol. 1, p. 100002, 2023.

[7] D. He, Y. Cai, S. Zhu, Z. Zhao, S. Chan, and M. Guizani, "A lightweight authentication and key exchange protocol with anonymity for IoT," *IEEE Transactions on Wireless Communications*, vol. 22, no. 11, pp. 7862–7872, 2023.

[8] S. Roy, M. H. Mahalat, and B. Sen, "Design and implementation of cost-effective end-to-end authentication protocol for PUF-enabled IoT devices," *IEEE Transactions on Emerging Topics in Computing*, vol. 13, pp. 1055 – 1067, 2025.

[9] P. Rosa, A. Souto, and J. Cecílio, "Light-SAE: A Lightweight Authentication Protocol for Large-Scale IoT Environments made with Constrained Devices," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 2428–2441, 2023.

[10] W. Yang, C. Hou, Y. Wang, Z. Zhang, X. Wang, and Y. Cao, "SAKMS: A Secure Authentication and Key Management Scheme for IETF 6TiSCH Industrial Wireless Networks based on Improved Elliptic-Curve Cryptography," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, pp. 3174–3188, 2024.

[11] S. Li, T. Zhang, B. Yu, and K. He, "A Provably Secure and Practical PUF-based End-to-End Mutual Authentication and Key Exchange Protocol for IoT," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 5487–5501, 2020.

[12] H. Weerasena and P. Mishra, "Lightweight Multicast Authentication in NoC-based SoCs," in *Proceedings of 2024 25th International Symposium on Quality Electronic Design (ISQED)*, pp. 1–8, IEEE, 2024.

[13] S. Das and M. Khatua, "Lightweight and decoupled distributed authentication for multihop IoT networks," in *Proceedings of IEEE 18th Int. Conf. Commun. Syst. Netw. (COMSNETS), Bengaluru, India*, 2026.

[14] Q. Fan, J. Chen, M. Shojafar, S. Kumari, and D. He, "SAKE*: A symmetric authenticated key exchange protocol with perfect forward secrecy for industrial Internet of Things," *IEEE transactions on industrial informatics*, vol. 18, no. 9, pp. 6424–6434, 2022.

[15] G. Yu, Q. Li, H. Mao, A. A. Abd El-Latif, and J. J. Rodrigues, "A forward-secure symmetric authenticated key exchange scheme with privacy preservation for Internet of Things applications," *IEEE Internet of Things Journal*, 2025.

[16] O. Alruwaili, M. Tanveer, S. A. Aldossari, S. Alanazi, and A. Armghan, "RAAF-MEC: Reliable and Anonymous Authentication Framework for IoT-enabled Mobile Edge Computing Environment," *Internet of Things*, vol. 29, p. 101459, 2025.

[17] R. B. Ponnuru, S. A. Kumar, M. Azab, and G. R. Alavalapati, "Baapfiot: Blockchain-assisted authentication protocol for fog-enabled internet of things environment," *IEEE Internet of Things Journal*, vol. 12, no. 11, pp. 15681–15696, 2025.

[18] Y. Zheng, Y. Cao, and C.-H. Chang, "Udhashing: Physical unclonable function-based user-device hash for endpoint authentication," *IEEE Transactions on Industrial Electronics*, vol. 66, no. 12, pp. 9559–9570, 2019.

[19] T. Idriss and M. Bayoumi, "Lightweight highly secure PUF protocol for mutual authentication and secret message exchange," in *Proceedings of 2017 IEEE International Conference on RFID Technology & Application (RFID-TA)*, pp. 214–219, IEEE, 2017.

[20] Y. Zheng, W. Liu, C. Gu, and C.-H. Chang, "PUF-based Mutual Authentication and Key Exchange Protocol for Peer-to-Peer IoT Applications," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 3299–3316, 2022.

[21] B. Blanchet, B. Smyth, V. Cheval, and M. Sylvestre, "ProVerif 2.00: Automatic Cryptographic Protocol Verifier, User Manual and Tutorial," tech. rep., INRIA, 2018. User Manual and Tutorial.

[22] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 2003.

[23] S. Yang, X. Zheng, G. Liu, and X. Wang, "IBA: A secure and efficient device-to-device interaction-based authentication scheme for internet of things," *Computer Communications*, vol. 200, pp. 171–181, 2023.